AgriCycle Test Specification

1. Introduction

AgriCycle is a mobile marketplace that connects suppliers of agricultural waste with buyers who want to reuse, recycle, or repurpose it. The system is implemented as an Android application developed in Android Studio and backed by Firebase Authentication and Firebase Realtime Database. Suppliers can publish waste listings, while buyers can browse listings, submit offers, or buy directly. The purpose of this document is to describe the testing strategy, test cases, and procedures that will be used to evaluate the quality of AgriCycle.

1.1 Goals and Objectives

The primary goal of testing AgriCycle is to identify defects and verify that the application meets its functional and non-functional requirements. In particular, the objectives are:

- • To detect logic and coding errors in key activities and adapters.
- • To ensure that the interface is usable for both suppliers and buyers.
- • To confirm that data is stored and retrieved correctly from Firebase.
- • To verify that the offer and deal lifecycle behaves as intended.
- • To assess the stability and performance of the app under normal usage.

1.2 Statement of Scope

Testing will focus on the core use cases implemented for the course project. These include user registration and login, listing creation by suppliers, browsing of listings by buyers, offer submission, direct purchase, and management of offers and accepted deals. The following levels of testing are in scope:

- • Unit Testing of individual activities and adapters.
- • Integration Testing of activity interactions and Firebase connections.
- • Validation Testing against the defined use cases UC1–UC3.
- • High-order System Testing including recovery, security, stress, and performance.

Other possible future features, such as advanced search filters, in-app messaging, or payment gateways, are not implemented in this phase and therefore are out of scope for this Test Specification.

1.3 Major Constraints

The AgriCycle project is subject to several constraints that influence the testing approach:

- • Fixed academic deadline defined by the course schedule.
- • Limited number of physical Android devices for testing.

- • Use of the free tier of Firebase, which limits large-scale load testing.
- • Team members balancing testing effort with other course responsibilities.

Because of these constraints, testing will emphasize correctness of core features and stability under typical loads rather than exhaustive performance benchmarking or large-scale scalability tests.

1.4 Assumptions and Dependencies

The test strategy is based on the following assumptions and dependencies:

- • Users have a stable internet connection while using the app.
- • Firebase Authentication and Realtime Database services are available.
- • Android devices meet the minimum OS and hardware requirements.
- • Test data (sample accounts and listings) can be created or reset as needed.

2. Test Plan

2.1 Software (SCIs) to be Tested

The software configuration items (SCIs) to be tested are grouped into interface components, backend components, help/documentation, and installation components.

2.1.1 Interface Components

- • SignActivity – Login screen for existing users.
- • SignUpActivity – Registration screen including role selection (buyer/supplier).
- • MainActivity – Dashboard that routes users to different functions based on role.
- • AddListingActivity – Form used by suppliers to create new waste listings.
- • ViewWasteListActivity – List screen showing listings, offers, or deals depending on context.
- • ProfileActivity – Screen for viewing basic user information and ratings.

2.1.2 Backend Components

- • Firebase Authentication (email/password login and registration).
- • Firebase Realtime Database node "users".
- • Firebase Realtime Database node "listing".
- • ListingAdapter – RecyclerView adapter that reads and writes listing data.

2.1.3 Help and Documentation

- • Labels, hints, and error messages within the Android UI.
- • Any supporting user guide or project presentation slides.

2.1.4 Installation Components

- • Installation of the APK file on supported Android devices.

- • Uninstallation of the application and removal of local data.

2.2 Testing Strategy

The testing strategy combines white-box and black-box techniques. Unit tests will examine individual classes and methods, while integration and validation tests will evaluate observable behaviour from the user's perspective. High-order tests will exercise the system under less common but realistic conditions such as network loss or heavy interaction.

2.2.1 Unit Testing

Unit testing will be applied mainly to activities and adapters that contain decision logic. Basis-path techniques will be used to cover important branches such as input validation and state transitions. Unit tests will be executed on the emulator and on at least one physical device.

2.2.2 Integration Testing

Integration testing will focus on the interaction between Android components and Firebase. Scenarios such as supplier and buyer coordinating an offer will be executed end-to-end to ensure that data flows correctly between screens and the backend.

2.2.3 Validation Testing

Validation testing will confirm that the implemented system satisfies the primary use cases:

- • UC1 – Supplier adds a waste listing.
- • UC2 – Buyer browses listings and submits offers or buys directly.
- • UC3 – Supplier reviews offers and manages accepted deals.

Each use case will be tested using normal flows and at least one alternative or error scenario.

2.2.4 High-Order System Testing

High-order testing will be performed on the fully integrated system. It includes recovery testing, basic security testing, stress testing with many listings, and rough performance checks. Alpha and informal beta testing will also be conducted within the class environment.

2.3 Testing Resources and Staffing

Testing will be conducted by all members of the AgriCycle team. The following roles describe how tasks are distributed; the same person may hold multiple roles depending on availability.

- • Rashid – Test Coordinator and Integration Tester.

- • Aria – Unit Testing Lead for activities.
- • Mohammed – Unit Testing Lead for adapters and Firebase logic.
- • Abdulla – System and Recovery Testing.
- • Yousef – Performance and Stress Testing.
- • Abdulqader & Mohd.Abdulla – Alpha/Beta Testing coordinator and feedback collection.

2.4 Test Work Products

The following work products will be created and maintained as part of the testing effort:

- • This Test Specification document describing plan and procedures.
- • Test Case tables for unit, integration, validation, and system tests.
- • Test Log containing records of executed tests and outcomes.
- • Bug/Defect Log summarizing discovered defects and their status.
- • Final Test Summary that highlights coverage and remaining risks.

2.5 Test Record Keeping

Test records will be maintained in a shared spreadsheet so that all team members can view and update results. Each executed test case will be logged with its identifier, execution date, tester name, and outcome (PASS/FAIL). When a defect is found, a reference to the corresponding bug entry will also be recorded.

2.6 Test Metrics

To monitor progress and quality, the following metrics will be collected during testing:

- • Number of test cases designed vs. number executed.
- • Percentage of test cases that passed on first execution.
- • Number of defects per component and per severity level.
- • Number of critical defects remaining at each milestone.
- • Informal coverage of major decision paths in core methods.

2.7 Testing Tools and Environment

Testing will be performed in the same environment used for development:

- • Android Studio and its emulator for different virtual devices.
- • Physical Android phones when available.
- • Firebase Console for monitoring authentication and database entries.
- • Spreadsheet software (Excel or Google Sheets) for recording test cases and logs.

2.8 Test Schedule

The test schedule is aligned with the overall project timeline and may be adjusted based on progress and feedback:

- • Week 1 – Prepare environment, create sample data, and design test cases.
- • Week 2 – Execute unit tests and fix high-priority defects.
- • Week 3 – Execute integration and validation tests; refine test data.
- • Final Days – Run high-order tests and regression tests; prepare test summary.

2.9 Risks and Contingencies

Key risks to the testing effort include late changes to requirements, technical problems with Firebase, and limited time near the deadline. Contingency actions include freezing new features before the final week and prioritizing tests that cover critical functionality such as login, listing creation, and offer management.

3.0 Test Procedure

3.1 Software (SCIs) to be Tested

The SCIs listed in the Test Plan will be tested according to the detailed procedures in this section. Each component describes stubs and drivers, representative test cases, the purpose of the tests, and expected results.

3.2 General Testing Steps

The general steps for executing tests are as follows:

1. 1. Ensure that the Firebase project is correctly configured and reachable.
2. 2. Create or reset test user accounts for supplier and buyer roles.
3. 3. Insert or clear sample listings in the database if required.
4. 4. Execute unit tests for the selected component on emulator and device.
5. 5. Execute integration and validation tests that involve multiple components.
6. 6. Record each test result and log any defects encountered.
7. 7. After fixes, re-run affected tests to confirm that defects are resolved.

3.3 Unit Test Cases

3.3.1 Component: SignActivity (Login)

SignActivity presents the login interface and performs authentication using Firebase.

Stubs and Drivers:

- • No explicit stubs. Firebase Authentication acts as the external service.

Representative Test Cases:

- • TC-SA-01 – Valid credentials; user is redirected to MainActivity.
- • TC-SA-02 – Invalid password; error message is displayed and user stays on login.
- • TC-SA-03 – Empty email or password; validation prevents login attempt.

Purpose:

To ensure that only valid users can access the system and that common error conditions are handled without crashes.

Expected Results:

Users with correct credentials reach the dashboard; all invalid inputs result in clear feedback and no unauthorized access.

3.3.2 Component: SignUpActivity (Registration)

SignUpActivity collects user details, including role selection, and registers new accounts.

Stubs and Drivers:

- • Uses Firebase Authentication and the "users" database node as external services.

Representative Test Cases:

- • TC-SU-01 – All fields valid; account is created and user record is stored.
- • TC-SU-02 – Missing required field (e.g., email); registration is blocked with a message.
- • TC-SU-03 – Weak password; Firebase rejects registration and message is shown.

Purpose:

To verify that registration enforces required fields, password rules, and role selection, and that corresponding user records are created correctly.

Expected Results:

Valid registrations produce new entries in both Authentication and the "users" node. Invalid data does not create accounts and produces clear error messages.

3.3.3 Component: AddListingActivity

AddListingActivity is responsible for collecting listing data from suppliers and saving it to Firebase.

Stubs and Drivers:

- • Real Firebase Realtime Database is used; supplier account acts as driver.

Representative Test Cases:

- • TC-AL-01 – All required fields valid; listing is saved under the correct supplier.
- • TC-AL-02 – Missing waste name; save is blocked with an error.
- • TC-AL-03 – Non-numeric or negative price; validation prevents saving.
- • TC-AL-04 – Network lost during save; app handles error without crashing.

Purpose:

To ensure that only valid listings are written to the database and that all input validation paths are exercised.

Expected Results:

Valid listings appear in the "listing" node with status set to available. Invalid inputs do not create listings and trigger appropriate error or toast messages.

3.3.4 Component: ListingAdapter (Buyer Mode)

In buyer mode, ListingAdapter exposes actions for making offers and buying directly.

Stubs and Drivers:

- • Driven from ViewWasteListActivity using a buyer test account.

Representative Test Cases:

- • TC-LB-01 – Make Offer with empty amount; prompt user to enter amount.
- • TC-LB-02 – Make Offer with valid amount; listing status becomes pending and buyer_id is set.
- • TC-LB-03 – Buy Direct and confirm; listing status becomes sold and buyer_id is set.
- • TC-LB-04 – Buy Direct and cancel; no changes to listing.

Purpose:

To validate that buyer actions modify listings correctly and that error conditions are handled gracefully.

Expected Results:

Database entries for offers and direct buys match the expected values; no invalid state transitions occur.

3.3.5 Component: ListingAdapter (Supplier Mode)

In supplier mode, ListingAdapter enables suppliers to accept or reject pending offers and view accepted deals.

Stubs and Drivers:

- • Driven from ViewWasteListActivity using a supplier test account.

Representative Test Cases:

- • TC-LS-01 – Accept a pending offer; listing status becomes sold and appears in accepted deals.
- • TC-LS-02 – Reject a pending offer; listing returns to available and buyer fields are cleared.
- • TC-LS-03 – View accepted deals; listings are read-only with deal information only.

Purpose:

To verify that the negotiation lifecycle is implemented correctly from the supplier's perspective.

Expected Results:

Accept and reject actions always lead to consistent and expected listing states; accepted deals cannot be edited.

3.3.6 Component: ProfileActivity (Ratings)

ProfileActivity displays basic user information and rating details and allows buyers to rate suppliers.

Stubs and Drivers:

- • Uses Firebase "users" node to retrieve and update rating values.

Representative Test Cases:

- • TC-PR-01 – View profile with existing rating; correct average is displayed.
- • TC-PR-02 – Submit new rating; rating totals and counts are updated.
- • TC-PR-03 – Open profile with no rating; appropriate default message is shown.

Purpose:

To ensure that rating information is correctly calculated and stored and that users receive accurate feedback.

Expected Results:

Rating values in Firebase always match the expected totals and averages after each rating action.

3.4 Integration Testing

Integration tests verify that components interact correctly when combined. The main focus is on complete workflows involving both roles and Firebase.

Integration Scenarios:

- • IT-01 – Supplier registers, logs in, creates a listing; buyer logs in and sees the listing.
- • IT-02 – Buyer submits an offer; supplier sees the pending offer and accepts it; both users see the deal.
- • IT-03 – Buyer performs direct buy; listing becomes sold and disappears from available listings.

Pass/Fail Criteria:

Integration testing is successful when all key workflows complete without crashes and produce correct database states and user-visible results.

3.5 Validation Testing

Validation testing ensures that the implemented system satisfies its intended use cases.

Validation Test Cases:

- • VT-UC1 – Execute UC1 (supplier adds listing) including one error path with missing data.
- • VT-UC2 – Execute UC2 (buyer browses, offers, or buys directly) including a cancelled offer.
- • VT-UC3 – Execute UC3 (supplier manages offers and deals) including accept and reject paths.

Pass/Fail Criteria:

Validation tests must pass for the project to be considered ready for demonstration. Any remaining defects must be minor and documented.

3.6 High-Order Testing

3.6.1 Recovery Testing

Recovery tests simulate network interruptions or Firebase outages during operations such as login or listing updates. The app should display an error or retry option rather than crashing.

3.6.2 Security Testing

Security tests verify that only authenticated users can access restricted screens and perform sensitive actions. Attempts to bypass login or modify data without proper permission should fail.

3.6.3 Stress Testing

Stress tests populate the database with many listings and offers, then perform rapid scrolling and repeated operations. The goal is to observe whether the app remains responsive and stable.

3.6.4 Performance Testing

Performance tests measure approximate response times for loading listings, submitting offers, and accepting deals. Observations are recorded to identify any obvious bottlenecks.

3.6.5 Alpha and Beta Testing

Alpha testing is performed by the project team, while beta testing is carried out informally by classmates who act as end-users. Feedback from both phases is captured in the bug log and used to plan final fixes.

4.0 Test Record Keeping and Test Log

4.1 Test Record Keeping

All executed tests will be recorded in a structured test log. Each record will include at least the date, type of test, module under test, coordinator or tester, and a brief description of the result. Defects will be cross-referenced with entries in the bug log.

4.2 Beta Tester Report Form

Name _____Mohammad Abdulla_____

Date _____26/11/2025_____

Defect Report ☑ Enhancement Request ☐

Module or Screen: _____Add Listing_____

Steps to Reproduce:

1. Open the AgriCycle app.
2. Log in as a farmer user.
3. Go to Add Listing screen.
4. Fill in Title, Category, and Price fields.
5. Leave the Quantity field empty.
6. Tap the Save Listing button.

Observed Behaviour:

- The app allows the listing to be submitted with an empty Quantity field and saves it to the database with a null/blank value.
- No validation message is shown to the user.

Expected Behaviour:

- The app should prevent submission if the Quantity field is empty.
- A clear validation message should appear, e.g., “Quantity is required and must be greater than 0.”
- The listing should only be saved when all required fields are filled correctly.

Severity: Low ☐ Medium ☑ High ☐

Additional Comments:

- Issue reproduced on multiple devices (Android 13 & 14).
- Affects data quality in the listings page and may confuse buyers when quantity is missing.
- Suggest adding front-end validation and backend check for required fields.

4.3 Test Log

	Test Type	System Module	Coordinator	Results
23-11	Unit Testing	SignActivity	Rashid	Valid and invalid login scenarios behave as expected.
23-11	Unit Testing	SignUpActivity	Aria	Registration validates required fields and stores new users.
23-11	Unit Testing	AddListingActivity	Mohammed	Listings with valid data are saved; invalid data is rejected.
24-11	Unit Testing	ListingAdapter (Buyer Mode)	Abdulla	Offer and direct buy transitions are correct.
24-11	Unit Testing	ListingAdapter (Supplier Mode)	Yousef	Accept and reject actions update listing status correctly.
24-11	Unit Testing	ProfileActivity	Abdulqader	Rating calculations and display are correct.
25-11	Integration Testing	Supplier–Buyer Offer Flow	Rashid	End-to-end offer creation and acceptance succeed.
25-11	Integration Testing	Direct Buy Flow	Aria	Direct buy marks listing as sold for both roles.
26-11	System Testing	Recovery / Network Loss	Mohammed	Network interruptions handled without crashes.
26-11	System Testing	Stress & Performance	Yousef	App remains responsive with many listings.